4-HOUR INSTRUCTOR-LED TRAINING · MICROSOFT AZURE

# LLM System Optimization & Deployment

Scalability · Lightweight Adapters · Enterprise Deployment on Azure

AI Engineers | ML Engineers | Data Scientists | Cloud Engineers | Software Engineers

**Ruthran Raghavan**
Chief AI Scientist
www.ruthranraghavan.com

AGENDA

# Presentation Agenda

Five sections across 4 hours of instructor-led content

**01** **Introduction**

Post-training lifecycle · Research vs Production · Deployment challenges

**02** **LLM System Optimization**

Latency · Throughput · GPU & Memory · Cost · Metrics

**03** **Scalability**

Vertical/Horizontal Scaling · Distributed Inference · KV Cache · Continuous Batching

**04** **Lightweight Adapters**

PEFT · LoRA · QLoRA · Prefix Tuning · Prompt Tuning · $IA^3$ · BitFit

**05** **Enterprise Deployment**

Azure AI Foundry · Security · MLOps · Case Studies · Governance

# What Happens After Training an LLM?

The journey from trained model to production system


### Training is 5% of the work

Deployment, monitoring, scaling and operations consume the majority of engineering effort in production AI systems.

### Production ≠ Research

Research cares about accuracy. Production cares about latency, cost, reliability, and serving thousands of concurrent users.

### The LLM Lifecycle is Continuous

Models must be updated, re-evaluated, monitored for drift, and re-deployed in a continuous loop — not shipped once.

# Research vs Production: A Critical Distinction

| Dimension | Research Environment | Production Environment |
|---|---|---|
| **Primary Goal** | Maximize accuracy / new techniques | Reliability, latency, cost, scalability |
| **Dataset** | Fixed benchmark datasets | Live user data, dynamic & evolving |
| **Hardware** | Single GPU / research cluster | Multi-GPU, multi-node, cloud infra |
| **Latency** | Not a priority (batch runs) | P50 < 500ms, P99 < 2s SLA |
| **Throughput** | 1 request at a time | Thousands of concurrent requests |
| **Failure mode** | Acceptable (retry & report) | Triggers incident response |
| **Reproducibility** | Required for papers | Best-effort; versioned deployments |
| **Cost** | Grant / lab budget | $/token, $/hour optimized KPIs |
| **Monitoring** | TensorBoard, loss curves | Prometheus, Grafana, APM, tracing |
| **Iteration** | Weeks per experiment | Continuous deployment, A/B testing |

Reference: Microsoft Azure Well-Architected Framework for AI · Anthropic Model Card Documentation · OpenAI Production Guidelines

# Why LLM Deployment is Uniquely Difficult

Eight compounding challenges that don't exist in traditional software

### ⚡ Extreme Model Size

GPT-3: 175B params, ~350GB fp16. Fitting in GPU memory requires model parallelism across multiple devices.

### ⏱️ Autoregressive Latency

Tokens generated one at a time. 500-token response at 30 tok/s = 16+ seconds without optimization.

### 💰 GPU Cost

A100 80GB costs ~$3/hr. A single model may need 8-16 GPUs. At scale this is millions/month.

### 🔀 Variable Request Length

Input/output lengths are unpredictable. Batching strategies must handle 10-token and 10,000-token requests simultaneously.

### 🔒 Security & Privacy

Models can memorize training data. Prompt injection, data leakage, and jailbreaks are active attack surfaces.

### 📊 Non-determinism

LLMs produce different outputs at temperature > 0. Testing, monitoring, and QA are fundamentally different.

### 🌍 Global Scale

Serving users across time zones with consistent SLAs requires multi-region deployment and geo-routing.

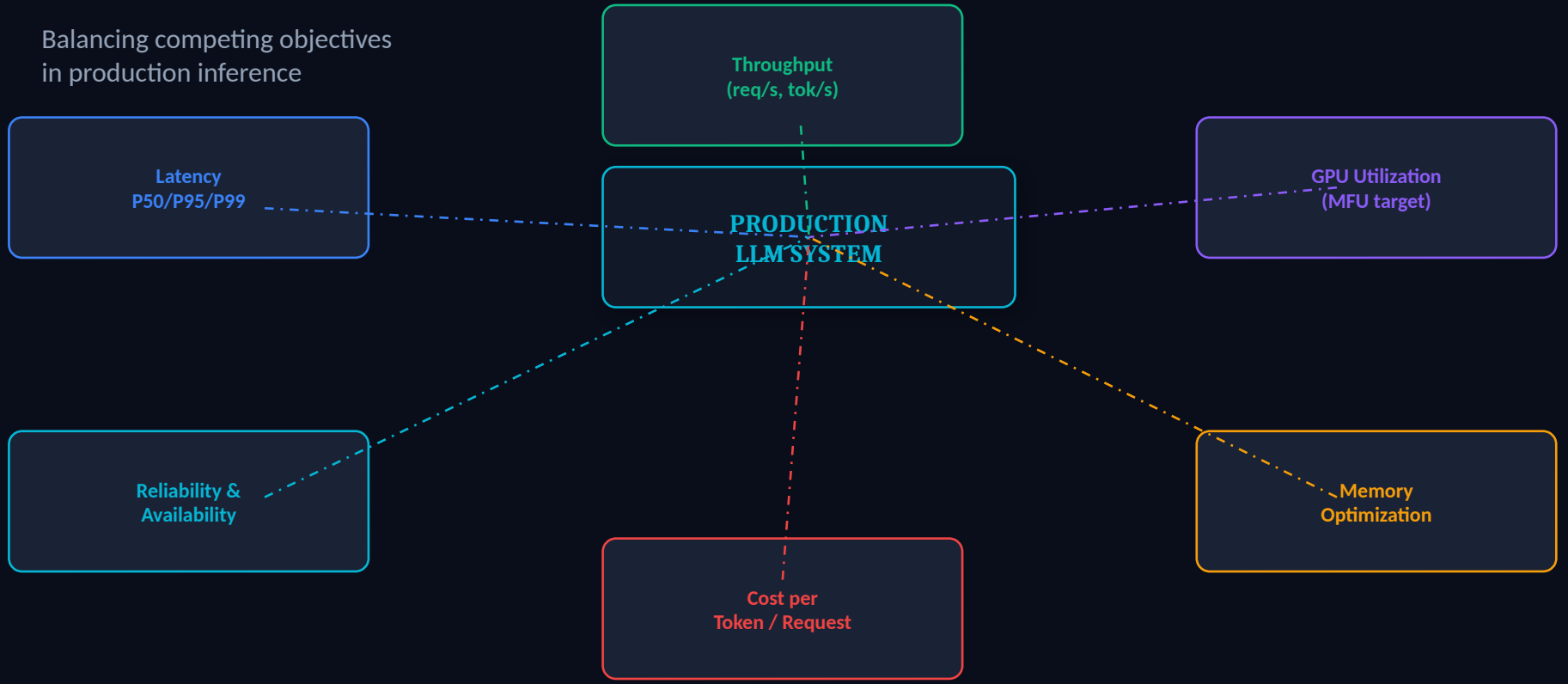
### 🔄 Continuous Evolution

Model versions, adapter weights, prompts, and safety filters all change independently and must be versioned.

SECTION 2: OPTIMIZATION

# LLM System Optimization: Key Dimensions

Balancing competing objectives in production inference




| | |
|---|---|
| < 300ms | Time to First Token TTFT target |
| 50-200 | Tokens / Second per request |
| > 50% | GPU MFU Model Flop Util. |
| $0.01–$5 | Cost / 1M Tokens production range |

# Latency, Throughput & GPU Optimization

Understanding the production inference bottleneck

## Latency Components

- TTFT (Time to First Token): KV cache miss, prefill phase
- TPOT (Time Per Output Token): Decode phase, memory bandwidth
- End-to-end latency: TTFT + N × TPOT

## Throughput Metrics

- Requests per second (RPS)
- Tokens per second (TPS) — total and per request
- Batch utilization rate

## GPU Optimization Techniques

| | |
|---|---|
| **Flash Attention:** | IO-aware exact attention, reduces HBM r/w by 5-20x. ArXiv: 2205.14135 |
| **Paged Attention:** | Non-contiguous KV cache via virtual memory pages (vLLM). Eliminates fragmentation. |
| **Quantization:** | INT8/INT4 weights reduce memory footprint 2-4x with <1% accuracy loss. GPTQ, AWQ. |
| **CUDA Kernel Fusion:** | Fuse elementwise ops to reduce kernel launch overhead and memory transactions. |
| **Tensor Parallelism:** | Split weight matrices across GPUs. Megatron-LM style tensor slicing for large models. |
| **Continuous Batching:** | Iteration-level scheduling. New requests join mid-generation. Increases GPU utilization 2-4x. |

References: Flash Attention (Dao et al., 2022) arXiv:2205.14135 · PagedAttention (Kwon et al., 2023) arXiv:2309.06180 · vLLM GitHub · NVIDIA TensorRT-LLM Docs

# Scalability: Vertical vs Horizontal

Scaling strategies for LLM inference workloads

## ↑ Vertical Scaling

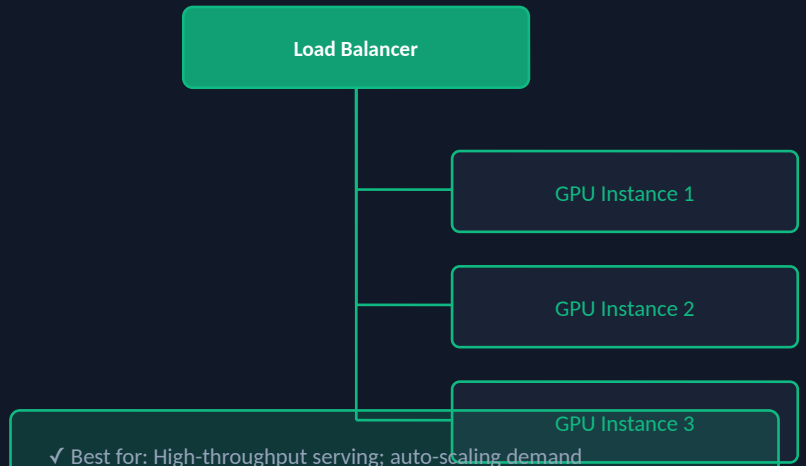
Add more powerful hardware to a single node

- A100 80GB → H100 80GB
- NVLink 600 GB/s interconnect
- More CPU cores + DRAM
- Faster NVMe storage

✓ Best for: Single large model that won't fit in smaller GPUs
✗ Limit: Physical hardware ceiling; expensive

## ↔ Horizontal Scaling

Add more nodes; distribute load across instances




✓ Best for: High-throughput serving; auto-scaling demand
✗ Limit: Model must fit in single node; network overhead

# Distributed Inference: Parallelism Strategies

Tensor, Pipeline, and Data Parallelism for LLM serving

## Tensor Parallelism

*Analogy: Splitting a single very wide table across multiple desks*

How: Weight matrices are sharded column-wise / row-wise across GPUs. Each GPU computes a partial result; results are aggregated via AllReduce.

When: When single layer doesn't fit in one GPU. Megatron-LM splits attention heads and FFN layers.

```
Memory per GPU ≈ Total Params / N_GPUs
```

Best Practice: Use Tensor||ism within a node (NVLink); combine with Pipeline across nodes for largest models

## Pipeline Parallelism

*Analogy: Assembly line: each station handles different transformer layers*

How: Different layers (transformer blocks) assigned to different GPUs. Micro-batching reduces pipeline bubble time. GPT-NeoX, Megatron use this.

When: For very deep models. Effective when layer count > GPU count. Requires careful micro-batch scheduling.

```
Bubble fraction ≈ (N_stages - 1) / M_microbatches
```

Best Practice: Set micro-batch count ≥ 4× pipeline stages to minimize bubble. Use gradient checkpointing

## Data Parallelism

*Analogy: Multiple cashiers serving different queues with identical registers*

How: Multiple identical model replicas each serve different request batches. No cross-GPU communication during forward pass.

When: When model fits on single GPU. Scale throughput linearly by adding replicas. Used in most horizontal scaling scenarios.

```
Throughput ≈ Single_replica_throughput × N_replicas
```

Best Practice: Add replicas behind a load balancer. Use auto-scaling groups on AKS / Azure ML

References: Megatron-LM (Shoeybi et al., 2019) arXiv:1909.08053 · PipelineParallelism (GPipe, Huang et al.) arXiv:1811.06965 · NVIDIA Megatron-LM GitHub

# KV Cache, Continuous Batching & Load Balancing

Core throughput optimization patterns for production LLM serving

## KV Cache (Key-Value Cache)

During autoregressive decoding, attention keys and values for all previous tokens are cached in GPU memory. Without caching, each decode step recomputes $O(seq^2)$ attention — prohibitively expensive at long contexts.

- Paged Attention (vLLM): virtual pages prevent fragmentation
- Memory: KV cache size = 2 × layers × heads × d_head × seq_len × batch × dtype_bytes
- For Llama-2 70B: ~1MB per token per batch element

## Continuous Batching (Iteration-Level Scheduling)

Traditional static batching waits for all requests in a batch to finish before accepting new ones. Continuous batching (Orca, 2022) inserts new requests at each iteration.

- GPU utilization improvement: 2-4x over static batching
- Key insight: shorter requests free GPU slots immediately
- Implemented in: vLLM, TGI (HuggingFace), TensorRT-LLM

## Load Balancing & Queue Management

### Round Robin

Distribute requests evenly. Simple but ignores GPU state.

### Least-Loaded

Route to replica with fewest active tokens in KV cache.

### Prefix-Aware Routing

Route same system-prompt requests to same replica to maximize KV prefix cache hits.

### Queue Management

Use token bucket / leaky bucket for rate limiting. Separate queues for streaming vs non-streaming.

### Auto-Scaling Trigger

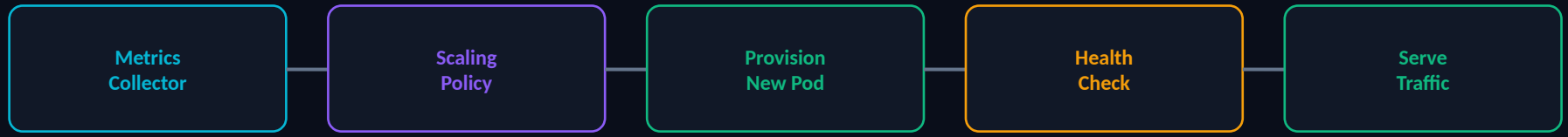
Scale out when queue depth > threshold or GPU utilization > 80% for >60s.

References: Orca (Yu et al., 2022) USENIX OSDI · vLLM (Kwon et al.) arXiv:2309.06180 · HuggingFace TGI Documentation

# Auto-Scaling, Streaming & High Availability

Production-grade reliability and responsiveness patterns

## Auto-Scaling Architecture




**Scale-Out Triggers:** GPU Utilization > 80% for 60s · Queue Depth > 100 requests · P99 Latency > 3s for 30s · Token rate > 90% capacity

### Server-Sent Events (SSE) Streaming

Stream tokens to client as generated. Reduces perceived latency dramatically. Client receives token stream via HTTP chunked transfer. Azure OpenAI stream: true parameter enables SSE. Handle backpressure at gateway layer.

### High Availability Patterns

Multi-AZ deployment: replicas across 3+ availability zones
Health probes: /health endpoint with 200 OK required
Circuit breaker: fail-fast on downstream errors
Graceful shutdown: drain active requests before pod termination
Target: 99.9% uptime = <8.7hr downtime/year

### Best Practice: Azure AKS HPA

Use Horizontal Pod Autoscaler with custom KEDA metrics. Scale on GPU utilization (DCGM Prometheus exporter) and queue depth (Azure Service Bus). Set minReplicas ≥ 2 for HA. Use Azure Spot VMs for cost optimization with preemption handling.

References: KEDA (Kubernetes Event-driven Autoscaling) docs · Azure AKS HPA documentation · Azure Monitor GPU Metrics with DCGM Exporter

# Parameter-Efficient Fine-Tuning (PEFT)

Fine-tune large models by updating only a fraction of parameters

**Why PEFT?**

Full fine-tuning of a 70B model requires 70B × 4 bytes = 280GB for weights alone + optimizer states (3× more for Adam). PEFT methods update <1% of parameters while achieving 95%+ of full fine-tuning performance.

| Full FT 70B | LoRA 70B | QLoRA 70B | Prompt Tuning | Prefix Tuning |
|---|---|---|---|---|
| GPU Req. | GPU Req. | GPU Req. | GPU Req. | GPU Req. |
| 8× A100 | 2× A100 | 1× A100 | 1× A100 | 1× A100 |
| Train Cost | Train Cost | Train Cost | Train Cost | Train Cost |
| $$$$$ | $$ | $ | $ | $ |
| Params | Params | Params | Params | Params |
| 100% | 0.1% | 0.1% | 0.001% | 0.01% |

**PEFT Method Taxonomy**

- Adapter Methods: LoRA, QLoRA, IA$^3$, AdaLoRA
- Prompt-based: Prompt Tuning, Prefix Tuning, P-Tuning v2
- Selective: BitFit (bias-only), Partial Layer Fine-Tuning

# LoRA & QLoRA: Low-Rank Adaptation

The most widely adopted PEFT methods in production

## LoRA (Low-Rank Adaptation)

Core Idea: Freeze pre-trained weights $W_0$. Approximate the weight update $\Delta W$ by decomposing into two low-rank matrices:

$$W = W_0 + \Delta W = W_0 + BA$$

$$B \in \mathbb{R}^{(d \times r)}, \quad A \in \mathbb{R}^{(r \times k)}, \quad \text{rank } r \ll \min(d,k)$$

- d and k = original weight matrix dimensions (e.g., 4096×4096)
- r = rank (typically 4, 8, 16, or 64). Only r << d parameters needed
- α = scaling factor (lora_alpha). Effective LR = α/r
- Trainable params: r×(d+k) vs original d×k. For r=8, d=k=4096: 65K vs 16.7M
- At inference: W+BA can be merged — zero overhead vs base model!
- Applied to: Q, K, V, O projection matrices in attention layers

## QLoRA

*QLoRA = 4-bit quantized base model + LoRA adapters in BFloat16*

**NF4 Quantization:**

4-bit NormalFloat: optimal for normally distributed weights. 2× compression vs INT8.

**Double Quantization:**

Quantize the quantization constants themselves. Saves ~0.37 bits/param extra.

**Paged Optimizers:**

Uses NVIDIA unified memory to page optimizer states from GPU ↔ CPU during gradient steps.

**Memory result:**

Fine-tune 65B model on single 48GB GPU. Full FT would need 8× A100 80GB.

**Accuracy trade-off:**

QLoRA achieves 99.3% of LoRA performance at 35% the memory cost (Dettmers et al., 2023).

References: LoRA (Hu et al., 2021) arXiv:2106.09685 · QLoRA (Dettmers et al., 2023) arXiv:2305.14314 · HuggingFace PEFT Library

# Other PEFT Methods: Prefix, Prompt, IA$^3$, BitFit

Complete comparison of lightweight adaptation strategies

## Prefix Tuning

Prepend trainable continuous vectors (prefix) to the Key and Value matrices of every attention layer. The model never sees the prefix as actual input tokens.

```
Params: prefix_len × 2 × n_layers × d_model
```

✓ Works for all sequence lengths ✗ Prefix occupies KV cache slots

Use when: NLP generation tasks. Better than Prompt Tuning for complex tasks. Prefixes not interpretable as natural language.

## Prompt Tuning

Prepend trainable soft tokens only to the input embedding layer (not to every layer). Simplest PEFT approach. Scales better with model size.

```
Params: prompt_len × d_embed (e.g., 100 × 768 = 76.8K)
```

✓ Single model, multiple tasks ✗ Underperforms on small models

Use when: Competitive with full FT for models >11B. Poor for smaller models. Easy to swap prompts per task.

## IA$^3$ (Infused Adapter)

Introduces learned rescaling vectors (l_k, l_v, l_ff) that multiply into attention keys, values, and FFN activations. Fewer params than LoRA.

```
Params per layer: 3 × d_model (vs LoRA: 2 × r × d_model)
```

✓ Extremely parameter-efficient ✗ Less flexible than LoRA

Use when: Few-shot scenarios. T-Few paper showed IA$^3$ outperforms LoRA on many-task benchmarks at 10× fewer parameters.

## BitFit (Bias Fine-Tuning)

Fine-tune only the bias vectors of all Linear layers. Freezes all weight matrices completely. Surprisingly effective for classification tasks.

```
Params: ~0.1% of model (bias vectors only)
```

✓ Trivially simple, extremely cheap ✗ Limited expressivity for generation

Use when: Quick adaptation with minimal resources. Best for classification, lower-level NLP tasks. Not suitable for generative tasks at high quality.

References: Prefix-Tuning (Li & Liang, 2021) arXiv:2101.00190 · Prompt Tuning (Lester et al., 2021) arXiv:2104.08691 · IA$^3$ (Liu et al., 2022) arXiv:2205.05638 · BitFit (Zaken et al., 2021) arXiv:2106.10199

# PEFT Methods: Complete Comparison

Decision guide for selecting the right fine-tuning strategy

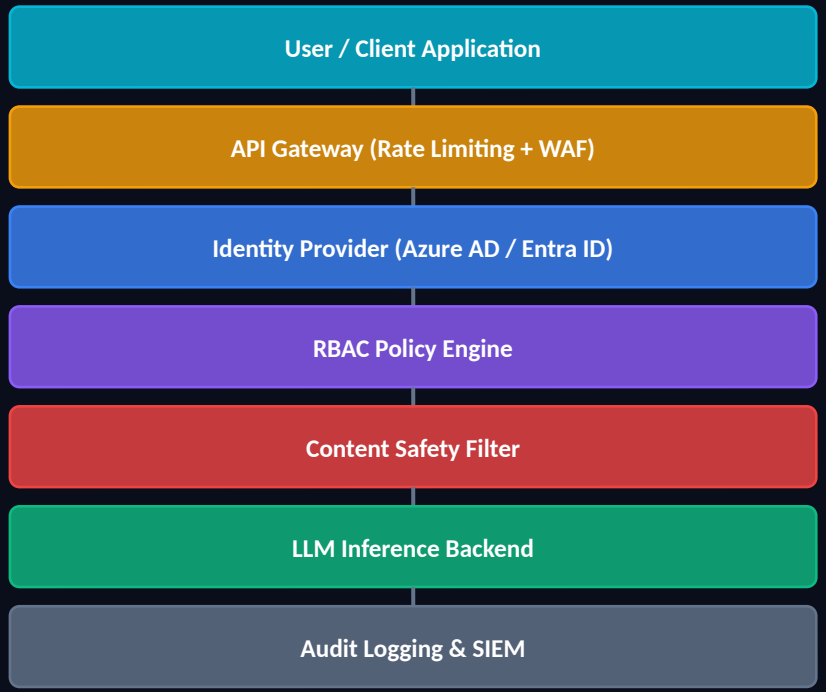
| Method | Trainable Params | GPU Memory | Inference Overhead | Best Use Case | Production Readine |
|---|---|---|---|---|---|
| **Full Fine-Tuning** | 100% | Very High (8×A100 for 70B) | None (merged) | Maximum task adaptation | ★★★★★ |
| **LoRA** | ~0.1% | Low (1–2×A100 for 70B) | None (mergeable) | General task fine-tuning | ★★★★★ |
| **QLoRA** | ~0.1% | Very Low (1×A100 for 65B) | Quantization overhead | Resource-constrained environments | ★★★★☆ |
| **Prefix Tuning** | ~0.01% | Very Low | Prefix in KV cache | Text generation, NLP tasks | ★★★☆☆ |
| **Prompt Tuning** | ~0.001% | Minimal | Extra input tokens | Large models (>11B), multi-task | ★★★☆☆ |
| **IA$^3$** | ~0.01% | Very Low | Element-wise multiply | Few-shot, many tasks on one model | ★★★★☆ |
| **BitFit** | ~0.1% | Minimal | None | Classification, quick experiments | ★★☆☆☆ |

Decision Rule: Start with LoRA (r=16). If memory-constrained → QLoRA. If no fine-tuning data → Prompt/Prefix Tuning. If many tasks on one model → IA$^3$.

SECTION 5: ENTERPRISE DEPLOYMENT

# Enterprise Security: Authentication, Authorization & RBAC

Zero-trust security architecture for production LLM systems




## Azure RBAC for LLM Systems

**AI Application Owner**

Full model management, deployment control, cost center access

**AI Model Deployer**

Deploy/update models, manage endpoints, view metrics

**AI API Consumer**

Call inference endpoints, read completions only

**AI Safety Reviewer**

View flagged content, update content filters, read audit logs

**AI Cost Analyst**

View usage metrics, cost dashboards — read only

Best Practices: Managed Identity (no keys in code) · Private Endpoints · Azure Key Vault for secrets · Conditional Access Policies

References: Azure RBAC Documentation (learn.microsoft.com) · Azure AI Services Security Baseline · Microsoft Entra ID for AI workloads

# MLOps / LLMOps: Monitoring & Observability

Continuous operations for production AI systems


## Metrics (RED Method)

- Rate: requests/second per endpoint
- Errors: 4xx/5xx rate, content filter triggers
- Duration: TTFT, TPOT, E2E latency percentiles
- Token metrics: input/output tokens, cost/request
- GPU: utilization, memory, temperature

## Logging & Tracing

- Structured JSON logs: request_id, user_id, model_version
- Distributed tracing: Azure App Insights, OpenTelemetry
- Prompt & completion logging (with PII masking)
- Audit logs: who called what endpoint, when
- Log retention: 30-90 days for compliance

## Alerting & Dashboards

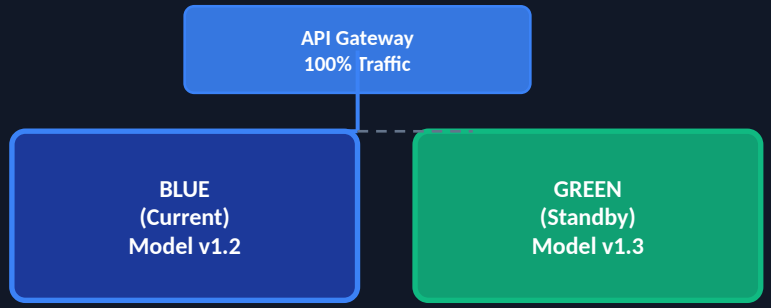
- P99 latency > SLA threshold → PagerDuty alert
- Error rate > 1% for 5 min → incident trigger
- GPU utilization < 20% for 30 min → scale-in
- Cost anomaly: spend > 2× baseline → alert
- Grafana/Power BI dashboards for stakeholders

References: Azure Monitor (learn.microsoft.com/azure/azure-monitor) · Azure Application Insights · OpenTelemetry Specification · Azure AI Foundry Monitoring

# Blue-Green & Canary Deployment for LLMs

Zero-downtime model updates and safe rollout strategies

## Blue-Green Deployment




1. Deploy new model (Green) alongside live (Blue)
2. Run smoke tests, benchmark, safety checks on Green
3. Switch API Gateway → Green in seconds
4. Blue becomes new standby (instant rollback)
5. If issues: flip back to Blue in < 30 seconds

## Canary Deployment

5%
Initial canary — validate with minimal users

20%
Expand if metrics healthy after 1h

50%
Gradual ramp; monitor error rates

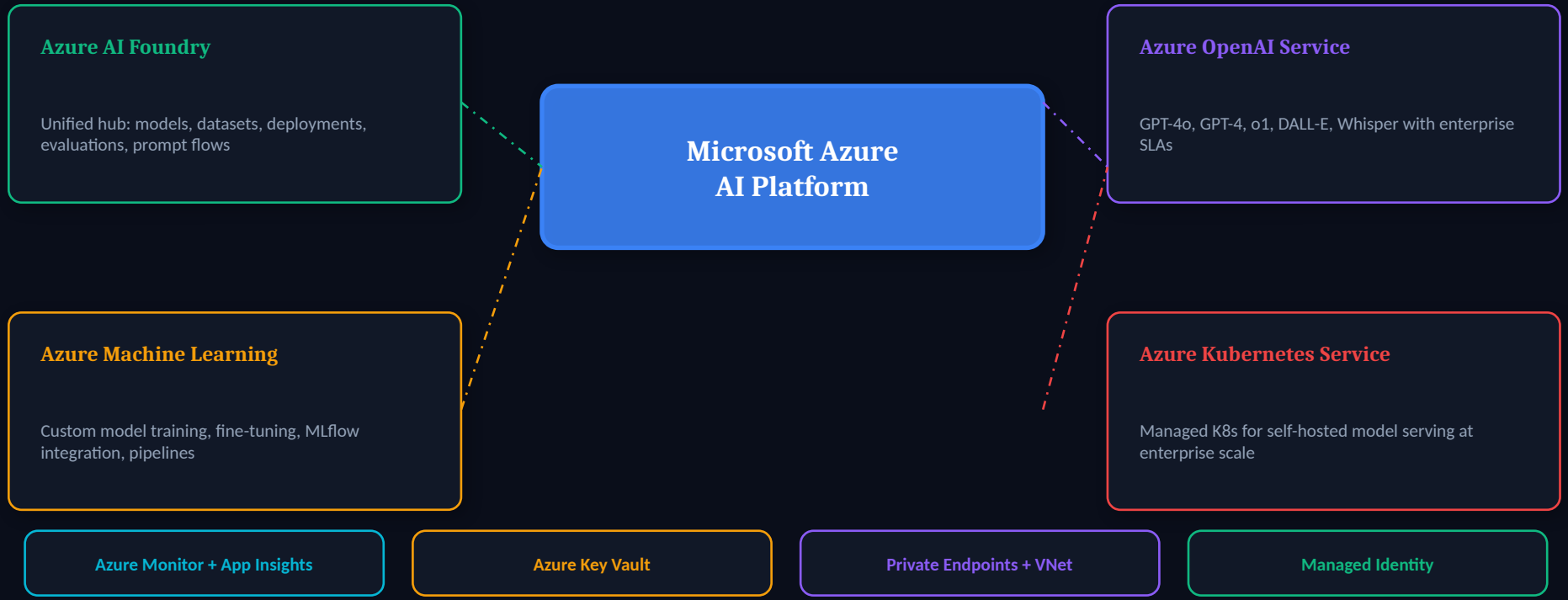
100%
Full rollout — decommission old version

Rollback trigger: error rate > 2% or P99 > 4s during canary phase → auto-rollback via pipeline gate

References: Azure Managed Online Endpoints Traffic Split · Azure AI Foundry Deployment Docs · MLOps on Azure (learn.microsoft.com)

# Microsoft Azure AI Platform Overview

Azure AI Foundry, Azure OpenAI Service & Azure Machine Learning




References: Azure AI Foundry Documentation (ai.azure.com) · Azure OpenAI Service (learn.microsoft.com/azure/cognitive-services/openai) · Azure Machine Learning Docs

SECTION 5: ENTERPRISE DEPLOYMENT

# Azure LLM Deployment Modes

Provisioned Throughput Units, Serverless, and Self-Hosted on AKS

## Provisioned Throughput Units (PTU)

*Reserved, dedicated capacity*

- Reserve dedicated GPU capacity in Azure datacenter
- Guaranteed throughput: e.g. 300 PTUs ≈ sustained 60 RPM for GPT-4
- Predictable latency SLA — eliminates noisy-neighbor effects
- Best for: high-volume, consistent production workloads
- Pricing: hourly committed capacity (monthly discount available)
- Minimum: 100 PTU purchase; scale in 100 PTU increments

## Serverless API (Pay-per-Token)

*On-demand, consumption-based*

- No capacity reservation; billed per 1K input/output tokens
- Zero management overhead — Microsoft manages all infra
- Best for: variable workloads, prototyping, bursty demand
- Automatic load balancing across Microsoft's global fleet
- Models: GPT-4o, GPT-4, GPT-3.5-Turbo, text-embedding-ada-002
- Rate limits: configurable TPM (Tokens Per Minute) per deployment

## Self-Hosted on AKS

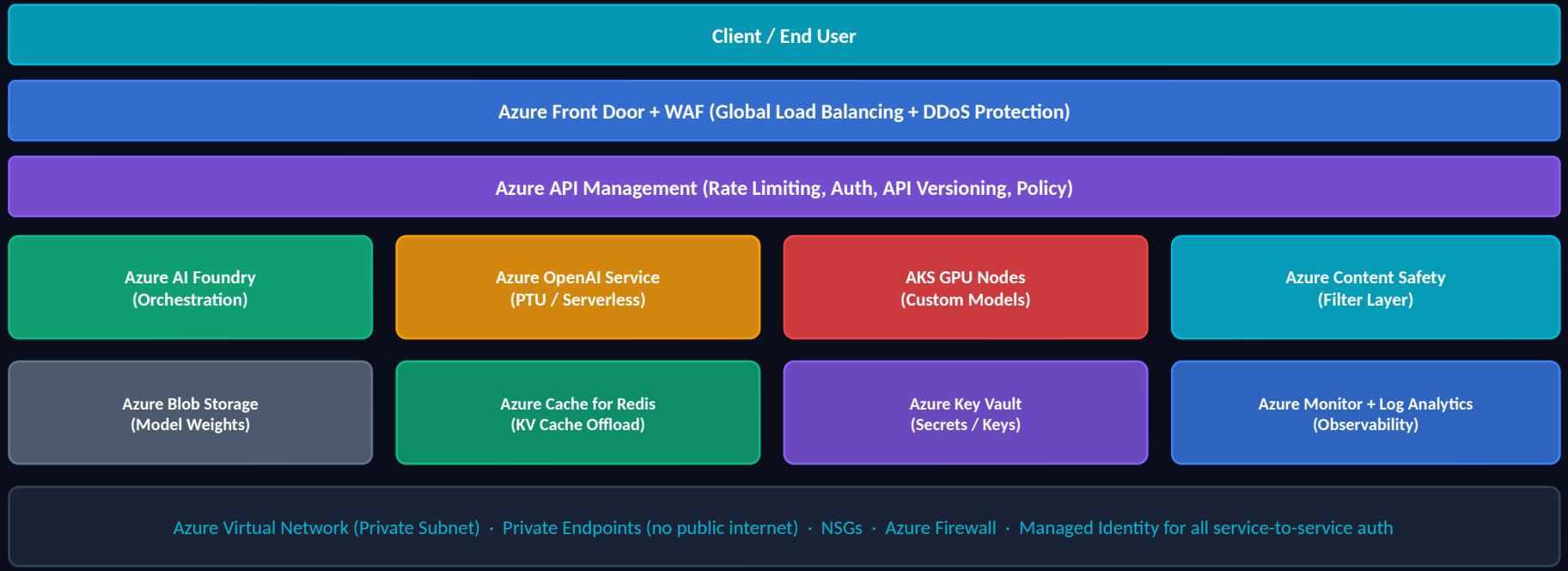
*Maximum control & customization*

- Deploy any open-source model: Llama 3, Mistral, Phi-3
- Full control over model weights, serving runtime, network
- Use NVIDIA TensorRT-LLM or vLLM on AKS GPU node pools
- Best for: fine-tuned models, data sovereignty, custom runtimes
- Integration: Azure Container Registry, ACR, Helm charts
- Cost: NC/ND series VMs; Spot for non-critical workloads

Choose: PTU for predictable high-volume · Serverless for variable demand · AKS for custom/open models + data sovereignty

# Azure Production LLM Architecture

End-to-end enterprise deployment reference architecture




References: Azure AI Landing Zone · Azure Architecture Center (docs.microsoft.com/azure/architecture) · Enterprise Azure OpenAI Service deployment guide

# Cost Optimization & Enterprise Governance

Controlling LLM costs and enforcing responsible AI policies

## Cost Optimization Strategies

| | |
|---|---|
| **Right-size Model:** | Use GPT-4o mini instead of GPT-4 where accuracy threshold permits. 10× cheaper/token. |
| **Prompt Compression:** | LLMLingua: compress prompts 3-6× with minimal quality loss. Saves input tokens. |
| **Response Caching:** | Semantic cache (Redis + embedding similarity). Cache hit rate 20-40% in practice. |
| **Batching:** | Batch non-real-time requests (document processing). Async queue + batch endpoint. |
| **PTU vs Serverless:** | PTU at high utilization (>60%) is 30-50% cheaper than pay-per-token serverless. |
| **Output Length Control:** | max_tokens parameter. Streaming + early stop. Reduces unused token generation cost. |
| **Azure Reservations:** | 1-year or 3-year committed use discounts on PTU. Up to 40% savings. |

## Enterprise Governance

### Responsible AI Framework

Microsoft RAI Principles: Fairness, Reliability, Privacy, Inclusiveness, Transparency, Accountability

### Content Safety

Azure AI Content Safety: Detect hate, violence, sexual content. Groundedness detection for RAG.

### Model Versioning

Immutable model artifacts in Azure ML Registry. Semantic versioning. Rollback capability.

### Data Governance

Microsoft Purview for data lineage. PII detection before prompts enter LLM. GDPR compliance.

### Disaster Recovery

Multi-region active-passive. Azure Traffic Manager failover. RTO < 15 min, RPO < 1 min targets.

References: Microsoft Responsible AI (microsoft.com/ai/responsible-ai) · Azure AI Content Safety · Azure Cost Management + Billing · LLMLingua (arXiv:2310.05736)

# Case Study: OpenAI — Large-Scale Inference Infrastructure

Publicly documented engineering practices

⚠ The following represents publicly documented OpenAI engineering practices. Internal implementation details not publicly disclosed are omitted.

## Infrastructure at Scale

- OpenAI operates one of the world's largest GPU clusters (Microsoft Azure partnership)
- Azure supercomputing infrastructure: 285,000 CPU cores, 10,000 GPUs per cluster (2020 disclosed)
- Multi-region API serving via Azure globally distributed infrastructure
- InfiniBand networking for GPU-to-GPU communication within training clusters

## API Serving & Load Balancing

- OpenAI API: Global load balancing across Azure regions
- Streaming responses via Server-Sent Events (SSE) for real-time token delivery
- Rate limiting per API key: TPM (tokens/min) and RPM (requests/min) tiers
- Usage tiers: free → tier 1-5 with increasing rate limits based on spend

## Safety & Monitoring Systems

- Moderation API: publicly available content policy enforcement endpoint
- System-level safeguards documented in model cards and usage policies
- Usage monitoring: automated detection of policy violations
- Red teaming: adversarial testing documented in GPT-4 technical report

## Enterprise API Tier

- ChatGPT Enterprise: SOC 2 compliance, enterprise SSO, no training on data
- Azure OpenAI Service: enterprise-grade SLA, private networking, RBAC
- API: Custom rate limits, fine-tuning endpoints (GPT-3.5, GPT-4)
- Batch API (2024): async processing for large workloads at 50% cost reduction

References: OpenAI System Card (GPT-4) · OpenAI API Documentation (platform.openai.com/docs) · OpenAI Microsoft Partnership Announcements (2023) · OpenAI Batch API Docs

# Case Study: Google — TPU Infrastructure & Vertex AI

Publicly documented Google AI deployment practices

⚠ The following represents publicly documented Google engineering practices. Internal implementation details not publicly disclosed are omitted.

## TPU Infrastructure

- TPU v4: 275 TFLOPS BF16 per chip; pods of 4,096 chips interconnected (exaflop-scale)
- Google's Pathways system: trains across thousands of TPUs as single accelerator
- Gemini Ultra trained on TPU v4 pods — documented in Gemini Technical Report (2023)
- Custom SparseCore in TPU v5e: accelerates embedding lookups for recommendation models

## Vertex AI & Gemini Serving

- Vertex AI: managed platform for LLM deployment, fine-tuning, and RAG pipelines
- Model Garden: 100+ models available including Gemini Pro/Ultra, open models
- Gemini API: available via Google AI Studio and Vertex AI with enterprise controls
- Serving: Tensor parallelism across TPUs; publicly documented in Pathways whitepaper

## Global Infrastructure

- 30+ Google Cloud regions globally for low-latency Vertex AI serving
- Multi-region deployments with automatic failover; 99.9% SLA for Vertex AI
- Dedicated hardware for enterprise customers (Google Cloud Distributed Cloud)
- Private Google Access: enterprise data stays within VPC, no public internet path

## Optimization & Enterprise

- Speculative decoding: draft model generates candidates; target model validates (Leviathan et al.)
- Context caching (Gemini 1.5): cache repeated context prefixes, charged at reduced rate
- Google Agentspace + NotebookLM: enterprise AI assistants on Gemini backend
- Responsible AI: SAIF (Secure AI Framework) published for enterprise guidance

References: Gemini Technical Report (google.com, 2023) · Google TPU v4 paper (ISCA 2023) · Vertex AI Documentation (cloud.google.com/vertex-ai) · Google SAIF Framework

# Case Study: Anthropic — Claude & Constitutional AI

Publicly documented Anthropic deployment and safety practices

⚠ The following represents publicly documented Anthropic engineering practices. Internal implementation details not publicly disclosed are omitted.

## Constitutional AI (CAI)

- Published technique: AI self-critiques responses using a set of constitutional principles
- Two stages: Supervised Learning (SL-CAI) + RLHF from AI feedback (RLAIF)
- Reduces need for human labelers on harmful content; documented in arXiv:2212.08073
- Powers Claude's safety properties: honesty, harmlessness, helpfulness (HHH)

## Multi-Cloud Deployment

- Amazon Bedrock: Claude models available via managed AWS API (Claude 3 family)
- Google Cloud Vertex AI: Claude models available in Model Garden
- Anthropic API: direct access at api.anthropic.com for developers
- Enterprise: Dedicated capacity available through Amazon Bedrock Provisioned Throughput

## Context Window & Efficiency

- Claude 3.5 Sonnet: 200K token context window (publicly documented)
- Prompt caching: cache system prompts to reduce latency and cost on repeated calls
- Streaming: SSE streaming supported across all API tiers
- Tool use / function calling: documented in Anthropic API reference

## Safety & Enterprise Controls

- System prompt confidentiality: users cannot extract system prompts by design
- Usage policies enforced at API layer: documented at anthropic.com/usage-policy
- SOC 2 Type II certified (enterprise tier)
- Interpretability research: mechanistic interpretability work published at anthropic.com/research

References: Constitutional AI (Bai et al., 2022) arXiv:2212.08073 · Claude Model Card (anthropic.com) · Anthropic API Documentation · Amazon Bedrock Claude Docs

# Industry Comparison: LLM Deployment Strategies

OpenAI vs Google vs Anthropic vs Azure — Publicly Documented Practices

| Dimension | OpenAI | Google (DeepMind) | Anthropic | Microsoft Azure |
|---|---|---|---|---|
| **Primary Hardware** | NVIDIA A100/H100 (Azure) | Custom TPU v4/v5 | NVIDIA (via AWS/GCP) | NVIDIA A100/H100 + AMD MI300X |
| **Model Serving** | Custom + Azure infra | Pathways + Vertex AI | API.anthropic.com + Bedrock | Azure OpenAI + AI Foundry |
| **Context Window** | GPT-4o: 128K tokens | Gemini 1.5 Pro: 1M tokens | Claude 3.5: 200K tokens | Dependent on hosted model |
| **Parallelism** | Not publicly disclosed | TPU Pod ICI fabric documented | Not publicly disclosed | Tensor/Pipeline (AKS + NCCL) |
| **Enterprise Access** | ChatGPT Enterprise + Azure OAI | Google Workspace + Vertex AI | Bedrock + GCP Vertex AI | Azure OpenAI PTU + Serverless |
| **Safety Layer** | Moderation API + RLHF | SAIF Framework + RAI | Constitutional AI (CAI) | Azure Content Safety + RAI |
| **Multi-Region** | Azure global regions | 30+ GCP regions | Via AWS & GCP | 60+ Azure regions globally |
| **Compliance** | SOC 2, HIPAA (enterprise) | SOC 2, HIPAA, FedRAMP | SOC 2 Type II | HIPAA, FedRAMP, ISO 27001, SOC |

Note: All information sourced from publicly available documentation. Internal infrastructure details not publicly disclosed are omitted from this comparison.

# Compliance, Model Versioning & Disaster Recovery

Production-grade operational requirements for regulated industries

## Compliance Frameworks

- HIPAA: BAA with Microsoft Azure. PHI in prompts requires encryption at rest + transit
- GDPR: Right to erasure — cannot delete from LLM weights. Keep PII out of prompts
- SOC 2 Type II: Azure AI services maintain audit logs for 90+ days
- FedRAMP High: Azure Government regions for US federal workloads
- ISO 27001: Information Security Management — Azure certified

## Model Versioning Strategy

- Immutable artifacts: model weights stored in Azure Blob with version tags
- Azure ML Model Registry: semantic versioning (major.minor.patch)
- Promotion gates: dev → staging → production with approval required
- Model metadata: training data version, hyperparams, eval scores tracked
- Rollback: re-deploy prior version in < 5 minutes via pipeline

## Disaster Recovery Architecture

**RTO Target:** < 15 minutes
Recovery Time Objective: time to restore service

**RPO Target:** < 1 minute
Recovery Point Objective: max acceptable data loss

**Pattern:** Active-Passive
Primary region active; secondary region hot standby

**Failover:** Azure Traffic Manager
DNS-based health-check driven failover

**Backup:** Geo-redundant storage
Model weights replicated to paired Azure region

**Testing:** Monthly DR drills
Automated failover test; RTO measured and tracked

References: Azure Compliance Documentation (learn.microsoft.com/azure/compliance) · Azure HIPAA/HITECH · Azure ML Model Registry Docs · Azure Traffic Manager Docs

# Production LLM Deployment Checklist

Engineering readiness gate for enterprise LLM systems

## Performance & Scaling

- [ ] Load test to 2× peak expected RPS
- [ ] P99 latency < SLA under load
- [ ] Auto-scaling policy validated
- [ ] KV cache hit rate > 40%

## Security & Access Control

- [ ] Managed Identity — no API keys in code
- [ ] Private Endpoint configured
- [ ] RBAC roles assigned (least privilege)
- [ ] Content Safety filter active

## Observability & Alerting

- [ ] Metrics exported to Azure Monitor
- [ ] Alerts set for latency + error rate
- [ ] Audit logging enabled (90-day retention)
- [ ] Dashboards live in Grafana/Power BI

## Deployment & Rollback

- [ ] Blue-Green or Canary strategy configured
- [ ] Rollback tested (< 5-min target)
- [ ] Model versioned in Azure ML Registry
- [ ] Deployment pipeline in CI/CD (ADO/GitHub)

## Cost & Governance

- [ ] Cost alerts configured in Azure Cost Mgmt
- [ ] PTU vs Serverless decision documented
- [ ] Responsible AI impact assessment completed
- [ ] Compliance requirements signed off (legal)

## Reliability & DR

- [ ] Multi-AZ deployment verified
- [ ] DR runbook written and tested
- [ ] Health probe endpoints returning 200 OK
- [ ] On-call rotation and escalation path defined

KEY TAKEAWAYS

# Key Takeaways: 10 Principles for LLM Production

Engineering wisdom for deploying LLMs at enterprise scale

**01 — Optimize for the bottleneck**
Profile first. TTFT vs TPOT vs memory vs network — fix the actual constraint, not the perceived one.

**02 — Continuous batching > static batching**
Always use iteration-level scheduling. 2-4× throughput improvement is non-negotiable for production.

**03 — Start with LoRA**
For task adaptation, LoRA with r=16 is the right default. Switch to QLoRA if GPU-constrained.

**04 — Security is zero-trust, always**
Managed Identity. Private Endpoints. No API keys in code. Assume breach as the starting posture.

**05 — Instrument everything**
TTFT, TPOT, GPU MFU, cost/token, error rate. You cannot optimize what you do not measure.

**06 — Design for rollback**
Blue-Green or Canary for every model update. A rollback that takes >5 minutes is too slow.

**07 — Cost is a first-class metric**
Cost/1K tokens is a KPI. PTU vs serverless at >60% utilization. Prompt compression saves 3-6×.

**08 — Use Azure AI Foundry as the hub**
Centralized model management, evaluations, deployments, and monitoring. Don't build from scratch.

**09 — Compliance is architectural**
HIPAA, GDPR, SOC 2 requirements must be in the design — not bolted on after deployment.

**10 — LLMOps is continuous**
Model updates, safety filters, adapter swaps, and cost optimization never stop. Build the pipeline.

# Thank You

## Questions & Discussion

**Ruthran Raghavan**

Chief AI Scientist · www.ruthranraghavan.com

### Key References & Further Reading

- Flash Attention: Dao et al. (2022) arXiv:2205.14135 · PagedAttention / vLLM: Kwon et al. (2023) arXiv:2309.06180
- LoRA: Hu et al. (2021) arXiv:2106.09685 · QLoRA: Dettmers et al. (2023) arXiv:2305.14314
- Constitutional AI: Bai et al. (2022) arXiv:2212.08073 · Megatron-LM: Shoeybi et al. arXiv:1909.08053
- Continuous Batching / Orca: Yu et al. (2022) USENIX OSDI · Gemini Tech Report: Google DeepMind (2023)
- Azure AI Foundry: ai.azure.com · Azure OpenAI: learn.microsoft.com/azure/cognitive-services/openai
- NVIDIA TensorRT-LLM: github.com/NVIDIA/TensorRT-LLM · HuggingFace PEFT: github.com/huggingface/peft
- Azure Kubernetes Service Docs: learn.microsoft.com/azure/aks · OpenTelemetry: opentelemetry.io